PRACTICAL- 13

PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

AIM / Problem Definition: Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

Tools / Technology Used: PHP 8.x, MySQL / MariaDB, JavaScript Fetch API, JSON, Browser Network DevTools, Apache / XAMPP, VS Code

Project Implementation & Architecture

Project Overview (StudentHub — PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload)

Following traditional server-side rendering in Practical 11, Practical 13 shifts to modern asynchronous decoupled web architecture. We developed a robust RESTful PHP JSON API endpoint (api-students.php) supporting standard HTTP verbs and paired it with a reactive single-page client interface (ajax-students.php). This architecture enables seamless data management without disruptive page reloads.

Technical Architecture & Workflow

RESTful API Endpoint (api-students.php): Implements standard HTTP methods (GET, POST, PUT, DELETE) emitting strict application/json headers with explicit HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error).

Asynchronous Fetch Client (ajax-students.php): Consumes the backend JSON API using modern JavaScript async/await and Fetch API. Modals handle creation and editing, updating the DOM dynamically without page refresh.

Live Debounced Search (Intermediate Extension): Implements a 300ms debounce timer on keyup inputs to throttle search queries against student_id, full_name, email, and department.

Defensive Deletion & Toast Notifications: Provides modal confirmation with optional soft-delete (status Inactive) or hard-delete, broadcasting animated floating toast alerts upon completion.

Integrated Network Inspector Panel: Renders an interactive real-time API inspector displaying method, endpoint URL, HTTP response status code, latency (ms), and formatted JSON payloads directly inside the browser.

Output Verification & Functional Screens

Output 1: AJAX Student CRUD Single-Page Interface

Interactive data table populated via GET /api-students.php on DOMContentLoaded, showing student IDs, full names, courses, CGPAs, and active/inactive status badges.

Output 2: Live Debounced Search in Action

Demonstration of typing search queries into the debounced input field, immediately narrowing the table from 21 records to matching student profiles within 300ms.

Output 3: Modal Add Student with Client & Server Validation (POST)

Add Student modal form submitting JSON payload to api-students.php via POST. Validates email formats and roll number patterns, returning 201 Created.

Output 4: Modal Edit Student & Dynamic Update (PUT)

Pre-populates existing student attributes via GET /api-students.php?id={id} and sends asynchronous PUT request, updating the row instantly.

Output 5: Delete Confirmation Modal & Soft/Hard Deletion (DELETE)

Defensive confirmation dialog with toggle for safe soft delete (status Inactive) or permanent removal via DELETE request.

Output 6: Real-Time Network Inspector Panel

Embedded developer inspector pane showing HTTP 200/201 response status, round-trip network latency (e.g. 14ms), and serialized JSON response structures.

Key Questions & Technical Analysis

Q1. Do PHP endpoints return valid JSON responses?

Yes. The endpoint strictly declares header("Content-Type: application/json; charset=utf-8"), serializes responses using json_encode() with JSON_UNESCAPED_UNICODE, and adheres to a standardized response envelope containing success flag, statusCode, message, data, and meta.

Q2. Are GET and POST requests handled properly?

Yes. The backend determines the request method via $_SERVER["REQUEST_METHOD"] (and supports _method override). GET retrieves lists with pagination or single objects by ID; POST creates records; PUT updates; and DELETE removes records safely.

Q3. Are loading, success, and error states shown on the frontend?

Yes. During asynchronous fetch calls, a semi-transparent loading spinner overlay is displayed. Upon resolution, high-contrast green toast alerts confirm success, red alerts highlight validation failures, and network errors are caught gracefully in try-catch blocks.

Q4. Is input validation performed before database operations?

Yes. Dual validation is enforced: client-side HTML5/regex patterns validate before transmission, while PHP backend enforces email validation, string sanitization, and duplicate roll number/email checks using prepared statements before database execution.

GitHub Repository Link

GitHub Repository URL: https://github.com/smit-2406/studenthub-website

Conclusion & Learning Outcomes

Practical 13 successfully implemented an asynchronous RESTful JSON API and Fetch-based single-page CRUD interface for the StudentHub Portal. The decoupled architecture eliminated full page refreshes, reduced network bandwidth, and provided an interactive user experience with debounced live search, modal operations, and transparent API status codes. This fulfills Course Outcomes CO3, CO4, and CO5.